

NOVA：面向工业推荐系统架构演化的验证感知智能体框架

Shaohua Liu*, Liang Fang*, Yilong Sun*, Shudong Huang*[†], Qingsong
Xiaoyang Chen, Dongqiang Liu, Chuangang Ma, Zhenzhen Chai, Henghu
Changyuan Cui, Zhangbin Zhu, Peng Chen, Wei Xu, Lei Xiao, Hai

腾讯公司

{jimmyshliu, franklfang, elonsun, ericdhuang, qsongluo, orions
dongcialiu, chuangangma, njzzchai, asherwang, justinquan, sh
pengchen, davidxu, shawnxiao, jerrickgu, zeus}@tencent

摘要

工业广告推荐系统通过架构修改得到持续改进，但生产迭代仍高度依赖专家，因为对模型拓扑、特征配置和交互模块的协同变更必须满足严格的接口、资源与服务约束。AutoML 受限于预定义搜索空间，而通用编码智能体验证的是可运行性，并非推荐系统特有的语义有效性。因此，可执行候选方案仍可能违反架构契约；同时，若缺乏对语义诊断与评估结果的结构化复用，就可能反复产生无效或无收益的修改。

我们提出 NOVA，一种具备验证意识的智能体框架，它在固定评估预算内，将生产架构修改组织为针对具体实现的多轮搜索。在每一轮中，NOVA 在生产约束下生成多个候选方案，剔除存在语义违规的方案，并对有效的剩余方案排序，以供本地测试与离线评估。跨轮次的轨迹记忆将语义诊断、本地测试结果和离线指标变化综合为修改方向与禁用模式，用以指导后续搜索。在自动化方法具有相同最大离线评估预算的条件下，NOVA 取得最高的有效通过率，在 ScaleUp 和 Literature-to-Production 任务上分别达到 53.3% 和 51.7%。在一个服务超过十亿用户的广告系统中，覆盖 5% 流量的生产 A/B 测试表明，选定的 Literature-to-Production 候选方案在三个主要 pCVR 目标上分别带来 +1.25%、

+1.70% 和 +2.02% 的 GMV 增益，同时绝对 pCVR 偏差分别相对降低 58.8%、66.7% 和 37.3%。

CCS CONCEPTS

• 信息系统 → 推荐系统.

KEYWORDS

推荐系统，架构修改，语义验证

1 引言

大规模推荐系统通过不断丰富特征与行为建模的架构取得了长足进展。FM 和 FFM [10, 16] 对显式特征交互进行建模；Wide & Deep、DeepFM 和 DCN [4, 6, 19] 学习高阶交互模式；DIN、DIEN 和 SIM [14, 26, 27] 则引入用户行为历史。近期面向生产的骨干网络，包括 RankMixer、TokenMixer-Large、MixFormer、HyFormer 和 OneTrans [7–9, 25, 28]，通过扩展模型容量并联合建模特征交互与行为序列，进一步提升了推荐质量。

尽管取得了这些进展，生产环境中的架构演进仍然缓慢、成本高昂，并且高度依赖专家判断。工程师

*这些作者对本工作贡献相同。

[†]通讯作者。

必须首先理解最新研究,并将一个有前景的架构思想转化为对模型拓扑、特征配置和交互模块的协同修改,同时保持与现有训练和服务接口的兼容性。随后,他们必须进行本地测试、运行离线训练、分析延迟得到的结果,并决定候选方案是否应进入在线测试。失败的候选方案会耗费大量工程时间和 GPU 资源,而在线 A/B 测试又受到业务风险的制约。因此,生产团队只能评估所有可行架构修改中的很小一部分,使整个过程难以规模化。

因此,要实现生产架构演进的规模化,就需要更有效地利用每一次试验。无效候选方案应在成本高昂的训练之前被淘汰,而尝试修改所产生的证据应予以保留,而非丢弃。标准本地测试可以暴露编译和形状错误,但有些候选方案虽然可以运行,却违反了推荐系统特有的架构约定。我们将其称为语义静默故障;例如,特征与 token 映射不匹配、缺失序列掩码、logit 融合路径不正确以及目标泄漏。它们不同于有效性故障,后者保持语义有效且可执行,却未能提升离线 AUC。语义验证可以在昂贵训练之前淘汰前一类故障,而后一类故障只能通过离线评估识别。

现有自动化方法无法完全应对这一场景。AutoML、HPO 和传统 NAS 搜索标量超参数或预定义算子空间,而生产升级往往需要在严格的接口与资源约束下进行跨模块协同修改。通用编码智能体利用执行反馈修复实现,而近期的智能体系统复用评估结果来改进后续方案。然而,在训练之前,它们通常不会验证推荐系统特有的语义,也不会在指导后续轮次时区分语义错误、本地测试失败和无效的离线结果(第2节)。

为应对这些挑战,我们提出 NOVA,这是一种具备验证感知能力的智能体框架,它将生产架构修改表述为固定评估预算和生产约束下的多轮候选搜索。与主要由执行和指标反馈驱动的实验循环不同,NOVA 将推荐系统特有的语义有效性设为显式决策阶段,并将其与任务内轨迹反馈相结合。在每一轮中,NOVA 生成多个候选修改,淘汰违反推荐系统特有架构语义的候选方案,并对有效的剩余方案排序,以进行本地测试和离线评估。由此产生的审查诊断、本地测试结果和离线指标变化会被记录在轨迹记忆中,并综合为

薄弱组件归因、修改方向和禁用模式,用以指导后续方案生成;同时,任务特定证据和历史证据支持候选排序。每项任务开始时,NOVA 会加载一个知识库快照,在整个搜索过程中提供固定的先验依据。

综上,本文作出以下贡献。

- **受生产约束的架构修改框架。**我们提出 NOVA,一种具备验证感知能力的智能体框架,它将生产架构修改组织为针对具体实现的多轮、多候选搜索,覆盖计算图、特征配置和结构参数。候选方案在生产约束下生成,并分别针对语义有效性、可执行性和离线有效性进行分阶段评估。
- **质量审查与任务内轨迹引导的搜索。**我们将质量审查与轨迹记忆、结构化反馈相结合,形成验证感知的搜索循环。质量审查在本地测试之前淘汰违反推荐架构约定的实现,随后利用任务局部轨迹历史和固定的历史知识库对语义有效的剩余方案排序。轨迹记忆记录语义诊断、本地测试结果和离线指标变化,并将其综合为薄弱组件归因、修改方向和禁用模式,以指导后续候选方案生成。
- **从离线搜索到在线部署的工业评估。**在相同的最大离线评估预算下,我们在两个生产任务上将 NOVA 与 AutoML 和编码智能体基线进行比较,并纳入一个回顾性人工参考。NOVA 在两个任务上均取得最高的有效通过率,在 ScaleUp 上达到 53.3%,在 Literature-to-Production 上达到 51.7%。在一次覆盖 5% 流量的生产 A/B 测试中,选定的 Literature-to-Production 候选方案使一个服务超过十亿用户的广告系统中三个主要目标的 GMV 提升 1.25–2.02%,并使 pCVR 绝对偏差相对降低 37.3–66.7%。

2 相关工作

NOVA 借鉴了三条研究路线:智能体式推荐优化、AutoML 与架构搜索,以及编码智能体或自动化 R&D 智能体。

智能体式推荐优化。近期系统表明,LLM 智能体能够参与生产推荐系统优化。YouTube 的 *Self-Evolving Recommendation System* [18]、AgenticRecTune [21] 和

Meta 的 Ranking Engineer Agent (REA) [11] 支持方案生成、配置调优和广告排序实验。NOVA 聚焦于生产推荐系统中的架构修改。它在离线训练前验证语义，并利用两类证据来源：固定的历史知识库和任务内轨迹记忆。每轮结束后，轨迹记忆记录语义诊断、本地测试结果和离线指标变化，以指导后续轮次中的候选方案生成、验证和排序。

AutoML、*NAS* 与 *HPO*。AutoML、神经架构搜索和超参数优化能够高效探索显式的超参数与架构空间 [1, 5, 12, 13, 15, 17]。尽管这些方法可以搜索预定义的拓扑选择，但它们不太适合对生产代码库进行协同修改，例如联合更新特征模式、交互模块、训练目标和服务接口。相比之下，NOVA 搜索可执行的跨模块修改方案，同时强制满足生产约束。

编码智能体与自动化 R&D 智能体。SWE-agent 和 OpenHands [20, 22] 等编码智能体，以及 ReAct 和 R&D-Agent [23, 24] 等推理与自动化 R&D 工作流，将代码编辑与迭代执行反馈相结合。这类反馈有助于调试和完成任务，但可运行的实现仍可能违反领域特定的架构语义。NOVA 增加了多源语义规则验证，并利用其报告、本地测试结果和离线指标变化，引导后续轮次避开无效或低效的修改。

3 问题形式化

给定生产推荐模型的初始架构 A_0 、用户请求 q 以及硬性生产约束 Ω ，目标是找到一个已实现的架构更新，使其在满足 Ω 的同时完成该请求。离线选择仅使用 q 中指定的目标。

架构状态与可行候选方案。在第 t 轮，架构状态为

$$A_t = (G_t, \phi_t, F_t). \quad (1)$$

其中， G_t 是计算图，包括交互模块、预测头和训练目标路径； F_t 是特征配置； ϕ_t 包含结构超参数。令 $\mathcal{E}$ 表示修改方案空间。方案 $e_{t,j} \in \mathcal{E}$ 可以修改其中任意组件。将 $e_{t,j}$ 应用于当前架构 A_t ，得到

$$\tilde{A}_{t,j} = \text{Apply}(A_t, e_{t,j}), \quad (2)$$

仅当候选方案满足 Ω 时才予以保留。这些约束涵盖张量形状与数据类型、特征可用性以及参数量/FLOPs 预算。满足这些约束的候选方案被视为生产可行，随后分别接受语义验证和本地测试，以检验语义有效性与可执行性。只有同时通过两个阶段的候选方案才会进入离线有效性评估。

反馈引导的架构搜索。每一轮都会生成多个生产可行的候选方案，剔除语义违规方案，并在评估一个候选方案之前对剩余方案排序。随后，所得语义诊断、本地测试结果和离线指标变化将被复用，以指导下一轮的候选方案生成、验证与排序。在固定离线评估预算下，搜索选择 A_{off}^* ，即观测到的离线指标最佳且生产可行、语义有效并可在本地执行的架构。之后，通过生产 A/B 测试，使用 GMV 和预测偏差对其进行评估；二者是验证指标，而非搜索目标。

4 NOVA 框架

4.1 智能体框架概述

图 1 展示了 NOVA 的端到端智能体框架。Main Agent 选择能力级别和执行模式，并协调从初始化到在线评估的七个阶段。LLM 智能体负责架构推理与审查，而确定性脚本则执行测试、训练和指标收集。每一轮离线过程连接三个核心模块：Architecture Modification 生成受生产约束的候选方案，Quality Review 对其进行筛选和排序，Trajectory Memory 记录其结果，并为后续搜索综合生成 Structured Feedback。

4.2 架构修改

在任务开始时，NOVA 解析用户请求 q ，并分析生产代码库以实例化 A_0 。对于“文献到生产”任务，材料理解模块还会进一步提取源论文的核心架构思想和实现细节，并将其纳入任务上下文。

在第 t 轮，方案设计使用任务请求 q 、当前架构 A_t 、来自轨迹记忆的结构化指导 z_t 以及固定先验 KB ，生成 K 个修改方案 $(e_{t,1}, \dots, e_{t,K})$ ，其来源为 $\mathcal{E}$ 。代码生成将每个方案应用于 A_t ，同时遵守生产约束 Ω ，得

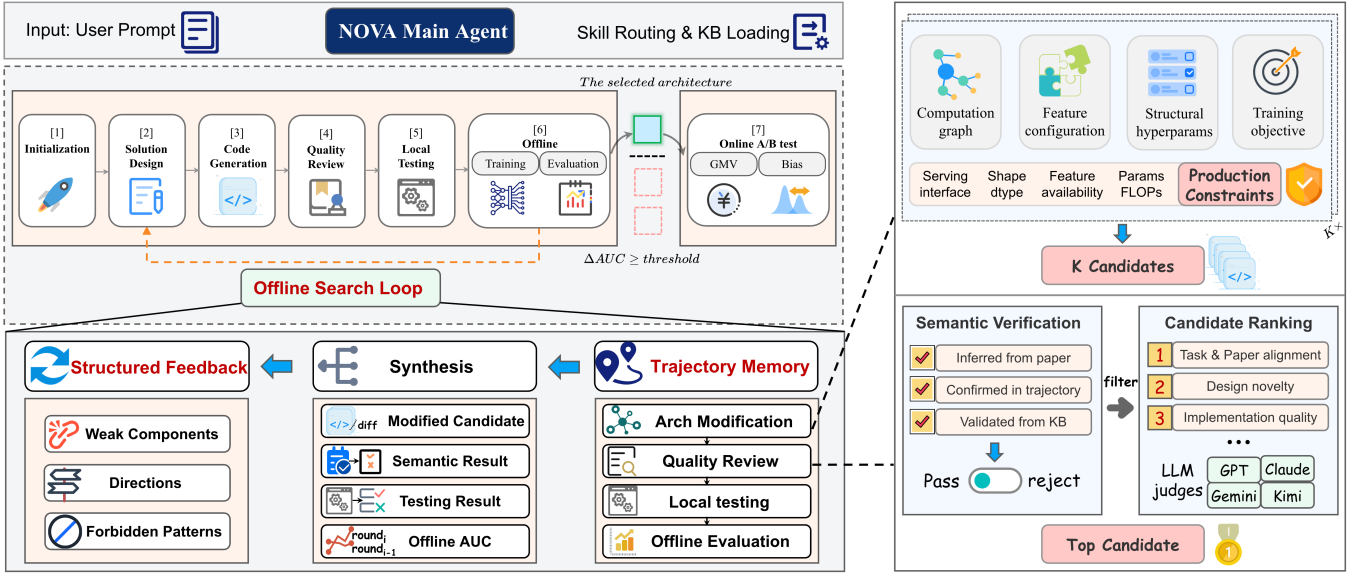


图 1: NOVA 验证感知搜索循环概览。在每一轮中，NOVA 生成 K 个受生产约束的候选修改，验证其架构语义，对有效候选进行排序，并通过本地测试和离线训练评估选中的候选。诊断信息与指标变化会记录在轨迹记忆中，并综合为供下一轮使用的结构化指导。离线找到的最佳候选将进入在线验证。

到 $\tilde{A}_{t,j}$ 。由此形成的候选集合为

$$C_t = \{c_{t,j} = (e_{t,j}, \tilde{A}_{t,j}) \mid j = 1, \dots, K\}. \quad (3)$$

生成 K 个候选可在每一轮中提供不同的修改路径，使后续质量审查阶段能够过滤无效实现，并从剩余候选中进行选择。

4.3 质量审查

质量审查包含两个阶段：语义验证拒绝违反适用语法规则的候选，证据引导排序则从有效的剩余候选中进行选择。

4.3.1 语义规则验证. 在任务的第 t 轮，NOVA 汇集当前生效的语义规则集 $\mathcal{L}_t$ ，其来源包括论文要求、此前轮次中确认的语义错误（例如表 3）以及来自 KB 的已验证规则。每条规则均记录其来源、作用域和条件。对于候选 $c_{t,j}$ ， $\mathcal{L}_{t,j} \subseteq \mathcal{L}_t$ 表示适用规则。NOVA 按如下方式执行语义验证：

$$r_{\text{sem},t,j} = V_{\text{sem}}(c_{t,j}; \mathcal{L}_{t,j}). \quad (4)$$

报告包含 $\{\text{pass}, \text{reject}\}$ 中的判定结果，以及用于定位被违反规则和代码位置的诊断信息。附录 A.2 给出了来自上述三类来源的代表性规则。

4.3.2 证据引导的候选排序. 只有语义有效的候选才会进入排序：

$$\mathcal{S}_t = \{c_{t,j} \in C_t \mid r_{\text{sem},t,j} \cdot \text{decision} = \text{pass}\}. \quad (5)$$

候选排序使用评估期间可获得的四个 LLM 家族的最新生产版本：GPT、Claude Opus、Gemini 和 Kimi。每个评审模型分别从五个维度对一个剩余候选独立评分：任务对齐度、适用时的论文对齐度、与 O_t 和 KB 中有效证据的一致性、设计新颖性以及实现质量。各评审模型权重相同，而五个维度采用固定的非均匀权重，并在所有任务和轮次中保持不变。令 $s_{t,j,d}^{(m)}$ 表示评审模型 m 对候选 $c_{t,j}$ 在维度 $d \in \mathcal{D}$ 上的评分，令 w_d 表示对应的维度权重。聚合得分为

$$\rho_{t,j} = \frac{1}{4} \sum_{m=1}^4 \sum_{d \in \mathcal{D}} w_d s_{t,j,d}^{(m)}. \quad (6)$$

NOVA 选择得分最高的剩余方案(记为 $c_{t,j_t^*} = (e_{t,j_t^*}, \tilde{A}_{t,j_t^*})$) 进行本地测试。只有本地测试通过后, 该方案才进入离线评估。

4.4 轨迹记忆与结构化反馈

轨迹记忆 O_t 包含第 t 轮开始时可用的候选记录。被选中的候选记录为

$$o_{t,j_t^*} = (c_{t,j_t^*}, r_{\text{sem},t,j_t^*}, r_{\text{loc},t}, \delta J_t), \quad \delta J_t = \tilde{J}_t^* - J_t. \quad (7)$$

其中, $J_t = J_{\text{offline}}(A_t)$, $\tilde{J}_t^* = J_{\text{offline}}(\tilde{A}_{t,j_t^*})$, 而 δJ_t 是相对于当前状态的增益。每个 $o_{t,j}$ 记录第 t 轮中的一个候选方案。被语义验证淘汰的候选方案存储为 $(c_{t,j}, r_{\text{sem},t,j}, \perp, \perp)$, 而被选中的候选方案还会记录其本地测试结果, 并在离线评估后记录 δJ_t 。这些记录被追加到累积历史 O_{t+1} 中; 语义有效但未被选中的候选方案则被省略。随后, NOVA 为下一轮综合生成反馈:

$$z_{t+1} = \text{Synthesize}(O_{t+1}) = (z_{t+1}^{\text{weak}}, z_{t+1}^{\text{dir}}, z_{t+1}^{\text{forbid}}). \quad (8)$$

这三个组成部分分别识别薄弱组件、推荐修改方向, 并记录后续轮次中应避免的无效修改。在综合过程中, 如果离线结果与预期修改不一致, 可能会触发对所选候选方案的检查。经确认的语义错误会被转换为一条语法规则, 并在后续所有轮次的验证中持续可用; 相应的候选方案和结果仍保留在 O_{t+1} 中。临时基础设施故障则被忽略。

4.5 端到端 NOVA 离线搜索循环

算法 1 将三个核心模块连接起来, 而不重复其内部机制。在每一轮中, NOVA 将累积轨迹 O_t 综合为结构化指导 z_t , 并用其生成候选集合 C_t 。随后, Quality Review 生成语义报告、过滤无效候选方案, 并对剩余方案排序。本地与离线结果被追加到 O_{t+1} 。NOVA 同时维护用于持续修改的工作架构 A_t , 以及用于最终选择的已观测最佳架构 A_{best} 。语义故障或本地故障不会改变工作架构, 而进入离线评估的候选方案将成为下一个工作状态。当达到改进阈值或用尽轮次预算时, 搜索返回 A_{best} 。

Algorithm 1 端到端 NOVA 离线搜索循环

Require: $q, A_0, \mathcal{E}, \Omega, KB, R_{\text{max}}, K, \tau, J_{\text{offline}}$, 执行模式 μ
Ensure: 用于在线验证的最佳可行离线架构 A_{off}^*

```

1:  $J_0 \leftarrow J_{\text{offline}}(A_0)$ ;  $A_{\text{best}}, J_{\text{best}} \leftarrow A_0, J_0$ ;  $O_0 \leftarrow \emptyset$ 
2: for  $t = 0$  至  $R_{\text{max}} - 1$  do
3:    $z_t \leftarrow \text{Synthesize}(O_t)$ 
4:    $(e_{t,1}, \dots, e_{t,K}) \leftarrow \text{SolutionDesign}(q, A_t, z_t, KB; \mathcal{E})$ 
5:    $C_t \leftarrow \{c_{t,j} = (e_{t,j}, \text{Apply}(A_t, e_{t,j}; \Omega))\}_{j=1}^K$ 
6:   计算  $r_{\text{sem},t,j}$  (依据式 (4)),  $\forall c_{t,j} \in C_t$ 
7:    $S_t \leftarrow \{c_{t,j} \in C_t \mid r_{\text{sem},t,j}.\text{decision} = \text{pass}\}$ 
8:    $O_{t+1} \leftarrow O_t \cup \{o_{t,j} = (c_{t,j}, r_{\text{sem},t,j}, \perp, \perp) \mid r_{\text{sem},t,j}.\text{decision} = \text{reject}\}$ 
9:   if  $S_t = \emptyset$  then
10:     $A_{t+1}, J_{t+1} \leftarrow A_t, J_t$ ; 继续
11:   end if
12:   计算  $\rho_{t,j}$  (依据式 (6)),  $\forall c_{t,j} \in S_t$ 
13:    $c_{t,j_t^*} = (e_{t,j_t^*}, \tilde{A}_{t,j_t^*}) \leftarrow \arg \max_{c_{t,j} \in S_t} \rho_{t,j}$ 
14:   if  $\mu = \text{Copilot}$  then
15:      $\text{HumanConfirm}(c_{t,j_t^*})$ 
16:   end if
17:    $r_{\text{loc},t} \leftarrow V_{\text{local}}(\tilde{A}_{t,j_t^*})$ 
18:   if  $r_{\text{loc},t}.\text{fail}$  then
19:      $o_{t,j_t^*} \leftarrow (c_{t,j_t^*}, r_{\text{sem},t,j_t^*}, r_{\text{loc},t}, \perp)$ 
20:      $O_{t+1} \leftarrow O_{t+1} \cup \{o_{t,j_t^*}\}$ 
21:      $A_{t+1}, J_{t+1} \leftarrow A_t, J_t$ ; 继续
22:   end if
23:    $\tilde{J}_t^* \leftarrow J_{\text{offline}}(\tilde{A}_{t,j_t^*})$ 
24:    $\delta J_t \leftarrow \tilde{J}_t^* - J_t$ 
25:    $o_{t,j_t^*} \leftarrow (c_{t,j_t^*}, r_{\text{sem},t,j_t^*}, r_{\text{loc},t}, \delta J_t)$ 
26:    $O_{t+1} \leftarrow O_{t+1} \cup \{o_{t,j_t^*}\}$ 
27:    $A_{t+1}, J_{t+1} \leftarrow \tilde{A}_{t,j_t^*}, \tilde{J}_t^*$ 
28:   if  $\tilde{J}_t^* > J_{\text{best}}$  then
29:      $A_{\text{best}}, J_{\text{best}} \leftarrow \tilde{A}_{t,j_t^*}, \tilde{J}_t^*$ 
30:   end if
31:   if  $J_{\text{best}} - J_0 \geq \tau$  then 终止
32:   end if
33: end for
34: return  $A_{\text{off}}^* \leftarrow A_{\text{best}}$ 

```

4.6 智能体框架实现

任务路由与控制。Main Agent 在各阶段之间路由修改方案、实现、审查报告、执行结果和结构化反馈, 同时控制 R_{max} , K 和 τ 。NOVA 为每项任务分配四种能力等级之一: L1 用于原子级结构调优, L2 用于具备约束感知的 ScaleUp, L3 用于 Literature-to-Production 迁移, L4 用于开放式架构创新。等级在任务期间保持不变, 并决定其工作流与技能。执行模式则依据已验证的技能覆盖范围和操作风险单独选择: AutoRun 自

主运行，而 Copilot 要求人工确认所选方案及其实现。各阶段之间仅交换与任务相关的结构化制品。

历史知识库快照。任务开始时，NOVA 从专用知识库加载任务特定快照 KB ，其中包含经专家验证的有效修改与失败模式。该快照为候选方案生成、语义验证和候选方案排序提供先验依据，并在当前任务期间保持不变。任务中验证的可复用规则可在任务结束后提交，并且只会出现在未来的快照中（附录 A.4）。

推理与执行分离。NOVA 将架构推理交给 LLM 智能体，将操作任务交给确定性脚本。LLM 智能体执行 Material Understanding、Solution Design、Code Generation、Quality Review 和结构化反馈综合。确定性脚本负责检索材料、运行本地测试与训练作业，以及收集指标和日志。这种分离在保持架构推理灵活性的同时，使常规执行更快且具有确定性。只有测试状态、指标和错误摘要等结构化结果会传回 LLM 智能体。

5 实验

5.1 研究问题

我们围绕以下四个研究问题评估 NOVA：

- **RQ1: 离线有效性。**在离线评估次数上限相同的条件下，NOVA 实现 AUC 正向提升的架构修改比例是否高于自动化基线？
- **RQ2: 组件贡献。**移除 NOVA 的各个组件，会如何影响其有效通过率以及不同评估阶段的失败率？
- **RQ3: 架构迁移。**NOVA 如何在生产推荐代码中适配并优化研究型架构？
- **RQ4: 在线影响。**离线选出的修改能否在生产 A/B 测试中提高在线 GMV 并降低 pCVR 偏差？

5.2 实验设置

任务。我们在两种架构修改设置下评估 NOVA。L2 ScaleUp 使用生产环境中的 RankMixer 风格 [28] 主干网络，搜索相互耦合的结构超参数，包括 token_cnt、token_dim 和 RankMixer 层数，同时将模型总规模控

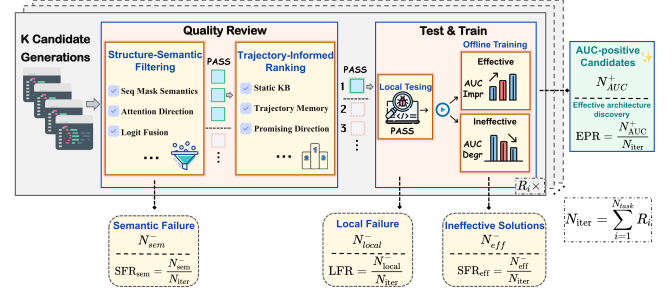


图 2: 任务级多轮候选漏斗。每项任务将该漏斗重复 R_i 轮。NOVA 和每个编码智能体基线在每轮的一次候选生成过程中生成 $K = 4$ 个候选方案，并至多对其中一个进行离线评估；所有比率均以 $N_{\text{iter}} = \sum_i R_i$ 为共同分母。

制在生产基线的 $\pm 10\%$ 范围内。**L3 Literature-to-Production** 评估一种方法能否在现有架构和训练流水线约束下，将源自文献的 token 交互模块（如 TokenMixer-Large [9] 和 MixFormer [7]）集成到生产主干网络中。

数据集。实验在一个从生产流量中采集的大规模工业广告推荐数据集上进行。训练语料覆盖一个月，包含十亿量级的用户-物品交互记录。每条记录均关联一千多个特征字段，涵盖序列和非序列信号。每种方法生成的每个候选模型都在同一数据集上从头训练。

预算与协议。所有依赖 LLM 的阶段均使用 **Claude Sonnet 4.6** [2]，但第 4.3 节所述 NOVA 的四模型排序集成除外。对于每种自动化方法，我们在每种设置下运行 $N_{\text{task}} = 20$ 个任务。L3 任务在 TokenMixer-Large 和 MixFormer 之间平均分配，各 10 个任务。每个任务最多运行 $R_{\text{max}} = 10$ 轮，并采用相同的提前停止阈值 τ 。因此， $N_{\text{iter}} = \sum_i R_i \leq 200$ ，但由于提前停止，不同方法之间存在细微差异。NOVA、其消融变体和编码智能体基线每轮生成 $K = 4$ 个候选。对于每个编码智能体基线，一个新的评估器仅依据任务说明、生产约束和候选内容，从随机打乱的候选中选出 Top-1 候选；NOVA 使用其质量审查，而 Optuna-TPE 遵循其原生的顺序提议机制。每轮最多离线评估一个候选，因此所有自动化方法拥有相同的最大离线评估预算。人工参考为回顾性结果，未进行预算匹配。

表 1: L2 ScaleUp 和 L3 Literature-to-Production 任务上的主要对比。EPR 是主要端到端指标；三种失败率用于定位评估漏斗中未成功的轮次。语义失败仅针对 L3 报告；“-”表示该方法不适用。人工结果行是回顾性的，未进行预算匹配。

方法	L2 ScaleUp			L3 Literature-to-Production			
	LFR	SFR _{eff}	EPR ↑	SFR _{sem}	LFR	SFR _{eff}	EPR ↑
人工参考	4.6%	46.2%	49.2%	16.9%	43.7%	8.5%	31.0%
OpenHands + Skills	66.7%	27.1%	6.3%	45.7%	15.2%	21.7%	17.4%
ReAct + Skills	62.5%	25.0%	12.5%	39.6%	27.1%	25.0%	8.3%
R&D-Agent + Skills	37.5%	33.3%	29.2%	19.6%	15.2%	37.0%	28.3%
Optuna-TPE	82.8%	12.1%	5.2%	—	—	—	—
NOVA	1.7%	45.0%	53.3%	13.3%	11.7%	23.3%	51.7%

指标。我们将离线质量报告为相对于基线的 AUC 增益（以百分点计）：

$$\Delta \text{AUC}_{\text{pp}}^{(0)}(A) = 100[\text{AUC}(A) - \text{AUC}(A_0)].$$

当 $\Delta \text{AUC}_{\text{pp}}^{(0)} \geq 0.10$ pp 时，已评估候选方案被视为 AUC 正向。若观测到的最佳基线相对增益达到 $\tau = 0.15$ pp，任务即终止。我们使用四个比率评估轮次级可靠性，并以 N_{iter} 作为共同分母：

$$\begin{aligned} \text{SFR}_{\text{sem}} &= \frac{N_{\text{sem}}^-}{N_{\text{iter}}}, & \text{LFR} &= \frac{N_{\text{local}}^-}{N_{\text{iter}}}, \\ \text{SFR}_{\text{eff}} &= \frac{N_{\text{eff}}^-}{N_{\text{iter}}}, & \text{EPR} &= \frac{N_{\text{AUC}}^+}{N_{\text{iter}}}. \end{aligned}$$

其中， N_{sem}^- 统计没有语义有效候选方案的轮次； N_{local}^- 统计所选候选方案未通过本地测试的轮次； N_{eff}^- 统计进入离线评估但低于 AUC 正向阈值的轮次； N_{AUC}^+ 统计产生 AUC 正向候选方案的轮次。在搜索过程中，NOVA 基于 LLM 的 V_{sem} 按照当前生效的语义规则筛选候选方案。为确保各方法的评估一致，所有报告的 SFR_{sem} 值均采用三名专家工程师的事后多数投票标签，他们会独立审计每个候选方案（附录 A.2）。对于每种自动化 L3 方法，一个轮次只有在计入 N_{sem}^- 的条件成立时，即全部 $K = 4$ 个候选方案均被标记为无效。对于回顾性的人工参考，其单个已评估候选方案适用相同标准。如图 2 所示，EPR 是主要的端到端指标，而三个阶段特定的失败率按照失败所处阶段对未成功轮次进行划分。

5.3 基线

我们将 NOVA 与编码智能体和 AutoML 基线进行比较，并纳入回顾性的人工参考。这是在匹配最大离线评估预算下进行的系统级比较；LLM 推理成本并未匹配。所有编码智能体基线均获得相同的可复用技能，用于论文理解、生产骨干分析和代码编辑。

- **人工参考。**我们使用相同的结果定义和 AUC 正向阈值，对可审计的资深工程师修改记录进行评分；但由于这些记录并非在匹配的搜索预算下产生，因此仅将其作为回顾性背景报告。
- **ReAct + Skills。**一个 ReAct [24] 智能体使用共享技能和重复执行反馈。它不使用 NOVA 的语义验证或任务内轨迹反馈。
- **OpenHands + Skills。**一个 OpenHands [20] 智能体使用共享技能和执行反馈。它不使用 NOVA 的语义验证或任务内轨迹反馈。
- **R&D-Agent + Skills。**我们实现了 R&D-Agent [23] 的预算匹配改编版本，该版本保留其 Researcher-Developer 组织形式并使用共享技能。Researcher 根据论文、代码和性能证据提出修改；Developer 实现这些修改并修复执行失败。该改编版本不包含 NOVA 的语义规则验证，也不包含其针对语义、本地测试和离线结果的任务内轨迹记忆。

- **Optuna-TPE**。Optuna-TPE [3] 在相同的任务级停止规则下搜索学习率、隐藏维度、块数、dropout、token 维度和 token 数量；它仅适用于 L2。

5.4 RQ1 的主要结果

表 1 显示, 与最强自动化基线 R&D-Agent 相比, NOVA 在 L2 上将 EPR 提高了 24.1 个百分点 (53.3% 对比 29.2%), 在 L3 上提高了 23.4 个百分点 (51.7% 对比 28.3%); 人工参考仅作为回顾性背景。

L2 ScaleUp。L2 ScaleUp 任务在一个 RankMixer 风格的生产骨干上进行。尽管它看似只是简单的参数调优, 但目标变量在结构上相互耦合; 例如, 受 token 化和交互设计约束, token_dim 必须能被 token_cnt 整除 [28]。NOVA 的 1.7% LFR 与自动化基线的 37.5%–82.8% 相比, 表明相较于黑盒搜索, 具备架构感知的约束处理具有优势。其 EPR 结果表明, 这种失败率下降转化为了更多 AUC 正向修改, 而非仅仅将失败推迟到离线评估阶段。

L3 Literature-to-Production。L3 评估在架构与训练流水线约束下, 将源自论文模块集成到生产骨干中的能力。与使用相同技能包的编码智能体基线相比, NOVA 以 51.7% 取得最高 EPR, 这表明仅凭共享技能不足以解释其系统级优势。RQ2 将进一步考察任务内轨迹引导迭代以及 NOVA 其他组件的贡献。

5.5 RQ2 的消融研究

为回答 RQ2, 我们在相同预算和评估协议下, 对 L3 设置中的 NOVA 进行消融实验。表 2 按架构修改、质量审查、任务内轨迹反馈和历史知识库对各变体进行分组。

架构修改。移除材料理解、方案设计或多候选生成后, EPR 分别降低 15.3、25.4 和 16.9 个百分点。 $K = 1$ 变体的轮次级 SFR_{sem} 无法与 $K = 4$ 变体直接比较, 因为语义失败要求一轮中的所有候选均无效。

质量审查。移除语义规则验证后, EPR 降低 30.5 个百分点; 移除证据引导的候选排序后, EPR 降低 40.3 个百分点。在排序消融中, 失败主要转移为离线无效 ($\text{SFR}_{\text{eff}} = 51.4\%$); 在验证消融中, 语义失败和有效性失败均有所增加。

轨迹记忆、结构化反馈与历史知识库。在保留历史知识库的情况下, 移除轨迹记忆和结构化反馈使 EPR 从 51.7% 降至 23.8%, 直接证明除共享技能外, 任务内轨迹引导的迭代也是一个主要贡献因素。在保留 O_t 和 z_t 的同时移除历史知识库快照, 会使 EPR 降低 37.4 个百分点。在两种变体中, 失败均向上游转移: SFR_{sem} 分别升至 42.9% 和 47.6%, 而移除 KB 还会使 LFR 升至 33.3%。

5.6 面向 RQ3 的生产代码修改案例研究

我们报告在 RankMixer 风格骨干上的两个 L3 案例。案例 1 使用轨迹反馈来改进迁移后的 TokenMixer-Large 模块 [9]。案例 2 通过门控多序列融合扩展 MixFormer [7]。

案例 1: 轨迹引导的辅助损失修正。目标 o_1 – o_3 属于同一个 pCVR 模型。目标是获得至少 +0.15 pp 的基线相对 AUC 增益, 该增益针对 o_1 , 且不能降低 o_2/o_3 。表 3 记录了六轮过程。

尽管第 3 轮通过了当前规则和本地测试, 但其有限的 o_1 增益以及 o_3 上的退化触发了代码检查。检查发现, o_1 专属辅助损失使用了错误目标的预测和标签索引, 而当前规则集中并未包含此错误。第 4 轮修正了索引, 在 AuxLoss 中屏蔽 o_3 , 并添加目标路由规则供后续轮次使用。第 5 轮在同时添加 token MLP 并提高损失权重后, 三个目标均出现退化。第 6 轮没有继续沿用该结果, 而是回滚至第 4 轮, 保留其索引和屏蔽修正, 仅重新调节 o_1 AuxLoss 权重, 从而在不降低 o_2/o_3 的情况下达到目标。

案例 2: 多序列 MixFormer 适配。图 3 将行为侧信息划分为产品属性、行为类型和行为时间序列。在每一层, 非序列 RankMixer 输出提供注意力查询和门控

表 2: L3 任务上的组件级消融。EPR 是主要的端到端指标；故障率展示了未成功轮次在各评估阶段的分布。所有 SFR_{sem} 值均采用独立专家评估。

变体	消融干预	SFR_{sem}	LFR	SFR_{eff}	EPR $\uparrow$
NOVA (完整)	完整系统	13.3%	11.7%	23.3%	51.7%
架构修改 (第 4.2 节)					
无材料理解	使用原始论文，不生成结构化理解产物	18.2%	13.6%	31.8%	36.4%
无方案设计	移除请求解读、主干网络分析和集成规划	36.8%	15.8%	21.1%	26.3%
无多候选生成	将 $K = 1$ 设为 $K = 4$	21.7%	30.4%	13.0%	34.8%
质量审查 (第 4.3 节)					
无语义规则验证	禁用搜索阶段的 V_{sem} ；所有满足约束的候选均进入排序	24.2%	15.2%	39.4%	21.2%
无证据引导的候选排序	从 S_t 中均匀选择，而不使用 $\rho_{t,j}$	20.0%	17.1%	51.4%	11.4%
轨迹记忆与结构化反馈 (第 4.4 节)					
无轨迹记忆与结构化反馈	移除 O_t 和 z_t ，同时保留 KB	42.9%	19.0%	14.3%	23.8%
历史知识库快照 (第 4.6 节)					
无知识库	将 $KB = \emptyset$ 设定为相应值，同时保留 O_t 和 z_t	47.6%	33.3%	4.8%	14.3%

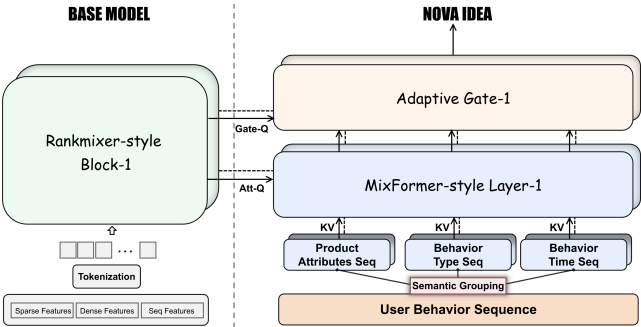


图 3: RankMixer (左) 与 NOVA 的 MixFormer 改编版本 (右)。语义分组产生三个行为序列；每一层使用来自非序列 RankMixer token 的查询，执行并行交叉注意力和门控残差融合。

查询；并行的 MixFormer 交叉注意力对三个序列进行建模，自适应门控则在残差更新之前融合其输出。相较于 MixFormer，该设计增加了语义子序列分解和门控多序列融合。

所选变体在前三个目标上的离线 AUC 分别提升 0.05、0.02 和 0.02 pp，参数量比 RankMixer 多 3.5%。

其增益仍低于 +0.15 pp 的晋级阈值，因此未进入在线 A/B 测试。

5.7 RQ4 的在线验证

案例 1 (表 3) 中第 6 轮的架构被选中用于在线验证。我们在 5% 的生产流量上，通过为期六天、按用户随机分组的 A/B 测试评估所得的 pCVR 模型。对于 pCVR 目标 o ，我们将预测转化率与观测转化率之比及其绝对偏差定义为

$$PCOC_o = \frac{\sum_{i \in \mathcal{D}_o} \hat{p}_{i,o}}{\sum_{i \in \mathcal{D}_o} y_{i,o}}, \quad \text{Bias}_o = |PCOC_o - 1|, \quad (9)$$

其中， $\mathcal{D}_o$ 为评估集， $\hat{p}_{i,o}$ 为预测 pCVR， $y_{i,o}$ 为观测转化指示变量，因此 $\sum_i y_{i,o}$ 为观测转化次数。使用 t 和 c 分别表示实验组和对照组，绝对偏差的相对降幅为 $1 - \text{Bias}_{o,t} / \text{Bias}_{o,c}$ 。表 4 报告了实验组相对于对照组的提升。

三个目标的 GMV 增益均为正且具有统计显著性，范围为 1.25% 至 2.02%；相对偏差降低幅度为 37.3%–66.7%。基于这些结果，我们计划推进该架构的全面部署。

表 3: 案例 1 的六轮轨迹。数值报告相对于基线的 $\Delta\text{AUC}_{\text{pp}}^{(0)}$ ，对应三个 pCVR 目标 $o_1/o_2/o_3$ 。目标为 $o_1 \geq +0.15$ pp，同时保持 o_2 和 o_3 非负。

轮次	架构修改	$\Delta\text{AUC (pp)}: o_1/o_2/o_3$	综合轨迹反馈
1	迁移 Mixing-and-Reverting、PerToken-SwiGLU 和 AuxLoss	+0.09/+0.06/+0.05	薄弱项: o_1 仍低于目标; 稠密参数超过主干网络的 $2\times$ 。方向: 在进一步修改前隔离 AuxLoss, 并恢复参数缩放。
2	移除 AuxLoss	+0.07/-0.03/-0.04	薄弱项: 移除 AuxLoss 会降低 o_2/o_3 , 而 o_1 仍远低于目标。方向: 以更低权重恢复该损失, 并增加 o_1 专属监督。
3	以 0.1 恢复 AuxLoss; 添加一个 o_1 专属辅助损失	+0.08/+0.05/-0.06	薄弱项: o_1 专属损失带来的增益很小, 表明其预测/标签索引可能不匹配; 恢复的 AuxLoss 会降低 o_3 。方向: 检查 o_1 的索引, 并在 AuxLoss 中屏蔽 o_3 。
4	修复 o_1 辅助索引; 在 AuxLoss 中屏蔽 o_3	+0.11/+0.02/-0.01	薄弱项: o_1 仍低于目标。方向: 添加一个 o_1 专属 token MLP 并增强监督。语义规则: 代码检查确认存在 o_1 索引不匹配; 每个辅助损失都必须路由至其设计所指定的目标。
5	添加一个 o_1 专属 token MLP; 提高 o_1 AuxLoss 权重	-0.24/-0.06/-0.06	薄弱项: MLP 和更高的损失权重使所有目标都出现退化。禁止项: 在当前任务中避免再次同时增加该 MLP 和损失权重。方向: 恢复到第 4 轮配置, 并重新调节 o_1 AuxLoss 权重。
6	恢复到第 4 轮; 保留修正后的 o_1 索引和 o_3 屏蔽, 并降低 o_1 AuxLoss 权重	+0.15/+0.02/+0.09	结果: 在不降低 o_2/o_3 的情况下达到目标; 停止。

表 4: 在 5% 生产流量上进行的为期六天、按用户随机分组的 A/B 测试结果。

pCVR 目标	GMV 提升	相对偏差 降低幅度
o_1	+1.25%	58.8%
o_2	+1.70%	66.7%
o_3	+2.02%	37.3%

5.8 部署经验与局限性

在生产环境中使用 NOVA 凸显了两个实践注意事项: 制品复用能够提高迭代效率, 而请求的具体程度会影响修改质量。

制品复用。NOVA 在任务之间复用缓存制品。Material Understanding 的输出和生产骨干分析分别以 arXiv URL 和骨干代码 ID 为键。对于重复使用已分析论文与骨干的任务, 检索这两类制品可使 Init 阶段 (Material & Backbone Understanding) 的单任务 LLM 成本降低 83.8% (附录表 5)。

请求具体程度。修改质量仍对请求的具体程度较为敏感。对于序列建模任务, 知识库包含多种适用于侧信息池化和序列-非序列融合的有效模式 (附录 A.4)。因此, 描述不充分的请求可能产生语义有效但偏离工程师意图的实现; 明确架构选择有助于提高一致性。

6 结论

我们提出 NOVA, 一个具备验证意识的智能体框架, 用于在部署约束下修改生产推荐架构。NOVA 生成多个候选方案, 在训练前剔除语义违规方案, 并使用轨迹记忆指导后续轮次。在 ScaleUp 和 Literature-to-Production 任务中, NOVA 在所评估的基线中取得了最高的有效通过率。其经离线选择的架构还在在线 A/B 测试的三个主要目标上提升了 GMV, 并降低了绝对 pCVR 偏差。

REFERENCES

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2623–2631.
- [2] Anthropic. 2026. Claude Sonnet 4.6. <https://www.anthropic.com/claude/sonnet>. Accessed: 2026-06-08.
- [3] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 24.
- [4] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ipsir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. 7–10.
- [5] Matthias Feurer, Aaron Klein, Katharina Eggenberger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and Robust Automated Machine Learning. In *Advances in Neural Information Processing*

Systems.

- [6] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine Based Neural Network for CTR Prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. 1725–1731.
- [7] Xu Huang, Hao Zhang, Zhifang Fan, Yunwen Huang, Zhuoxing Wei, Zheng Chai, Jinan Ni, Yuchao Zheng, and Qiwei Chen. 2026. MixFormer: Co-Scaling Up Dense and Sequence in Industrial Recommenders. *arXiv preprint arXiv:2602.14110* (2026).
- [8] Yunwen Huang, Shiyong Hong, Xijun Xiao, Jinqiu Jin, Xuanyuan Luo, Zhe Wang, Zheng Chai, Shikang Wu, Yuchao Zheng, and Jingjian Lin. 2026. HyFormer: Revisiting the Roles of Sequence Modeling and Feature Interaction in CTR Prediction. *arXiv preprint arXiv:2601.12681* (2026).
- [9] Yuchen Jiang, Jie Zhu, Xintian Han, Hui Lu, Kunmin Bai, Mingyu Yang, Shikang Wu, Ruihao Zhang, Wenlin Zhao, Shipeng Bai, et al. 2026. TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders. *arXiv preprint arXiv:2602.06563* (2026).
- [10] Yuchin Juan, Yong Zhuang, Wei-Sheng Chin, and Chih-Jen Lin. 2016. Field-aware Factorization Machines for CTR Prediction. In *Proceedings of the 10th ACM Conference on Recommender Systems*. 43–50.
- [11] Ashwin Kumar, Erwin Gao, Matan Levi, Sheela Yadawad, Sherman Wong, Sneha Iyer, and Vinodh Kumar Sunkara. 2026. Ranking Engineer Agent (REA): The Autonomous AI Agent Accelerating Meta’s Ads Ranking Innovation. Meta Engineering Blog. <https://engineering.fb.com/2026/03/17/developer-tools/ranking-engineer-agent-rea-autonomous-ai-system-accelerating-meta-ads-ranking-innovation/> Accessed: 2026-05-28.
- [12] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2019. DARTS: Differentiable Architecture Search. In *International Conference on Learning Representations*.
- [13] Hieu Pham, Melody Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. 2018. Efficient Neural Architecture Search via Parameter Sharing. In *Proceedings of the 35th International Conference on Machine Learning*. 4095–4104.
- [14] Qi Pi, Weijie Bian, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based Interest Model for Lifelong User Behavior Sequence Modeling in Click-Through Rate Prediction. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. 2685–2692.
- [15] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized Evolution for Image Classifier Architecture Search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4780–4789.
- [16] Steffen Rendle. 2010. Factorization Machines. In *Proceedings of the 2010 IEEE International Conference on Data Mining*. 995–1000.
- [17] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2013. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 847–855.
- [18] Haochen Wang, Yi Wu, Daryl Chang, Li Wei, and Lukasz Heldt. 2026. Self-evolving recommendation system: End-to-end autonomous model optimization with LLM agents. *arXiv preprint arXiv:2602.10226* (2026).
- [19] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the ADKDD’17*. 1–7.
- [20] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations*.
- [21] Xidong Wu, Yue Zhuan, Ruoqiao Wei, Hangxin Chen, Di Bai, Jintao Liu, Xinyi Wang, Xue Wang, Luoshu Wang, and Xinwu Cheng. 2026. AgentRecTune: Multi-Agent with Self-Evolving Skillhub for Recommendation System Optimization. *arXiv preprint arXiv:2604.26969* (2026).
- [22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37.
- [23] Xu Yang, Xiao Yang, Shikai Fang, Bowen Xian, Yuante Li, Jian Wang, Minrui Xu, Haoran Pan, Xinpeng Hong, Weiqing Liu, Yelong Shen, Weizhu Chen, and Jiang Bian. 2025. R&D-Agent: Automating Data-Driven AI Solution Building Through LLM-Powered Automated Research, Development, and Evolution. *arXiv preprint arXiv:2505.14738* (2025).
- [24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*.
- [25] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2026. Onetrans: Unified feature interaction and sequence modeling with one transformer in industrial recommender. In *Proceedings of the ACM Web Conference 2026*. 8162–8170.
- [26] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep Interest Evolution Network for Click-Through Rate Prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 5941–5948.
- [27] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1059–1068.
- [28] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.

A 附录：实现、审计与可复现性细节

本附录包含四个部分。我们报告各阶段的 LLM 使用量与成本,通过代表性语义规则及其审计轨迹展示 Quality Review 机制,给出可在机制层面揭示 NOVA 且无需逐字发布专有生产提示词的可复现性制品,并描述其知识库的组织方式和增量构建过程。

A.1 各阶段 LLM 使用量与成本

表 5 报告 NOVA 在各 workflow 阶段的平均 Claude API 使用量与成本。

token 使用量和成本主要由 Code Generation 与 **Initialization** 阶段产生:前者在信息最丰富的生产上下文中生成规模最大的编辑,后者则载入源论文和已部署代码库。Local Testing 与 Offline Training 主要由确定性脚本执行;其中少量 LLM 使用仅用于解释精简的状态与错误摘要。离线指标以确定性方式计算;Offline Evaluation 所报告的 LLM 使用量来自结果解释与结构化反馈综合。Main Agent 只负责编排,从不直接调用 LLM。

表 5: NOVA 各阶段的 Claude API 使用情况。所有数值均为在单个任务的全部工具调用轮次中累计得到的每任务平均值。初始化表示任务开始时对材料和生产主干网络的理解。未缓存输入 Token 和缓存读取输入 Token 是 Claude API 的两个输入字段,其中缓存读取输入的计费约为基础输入费率的 10%。成本根据报告的 token 用量和 token 单价计算。

阶段	Token 用量			成本
	未缓存	缓存读取	输出	
初始化	797.7K	14.45M	88.7K	\$8.17
初始化 + 产物复用	137.2K	2.24M	14.7K	\$1.32
方案设计	570.3K	6.85M	52.0K	\$4.68
代码生成	903.6K	17.13M	90.0K	\$9.30
质量审查	520.0K	9.20M	55.0K	\$4.80
本地测试	120.0K	1.60M	8.0K	\$1.05
离线训练	40.0K	0.35M	2.5K	\$0.35
离线评估	98.0K	1.34M	6.3K	\$0.79

A.2 代表性语义规则与审计轨迹

如第 4.3 节所述, V_{sem} 根据论文要求、由先前轮次中已确认的语义错误推导出的规则,以及来自 KB 的已验证规则检查候选方案。表 6 给出了每个来源的一条代表性规则。前三列实例化了第 4.3 节定义的规则字段;最后一列展示条件被违反时发出的诊断。这些示例用于说明规则结构,不用于估算所报告的语义故障率。

论文要求和 KB 规则在任务开始时加载。在当前任务中,从已确认错误中得到的规则会应用于所有后续轮次。 KB 快照在整个任务期间保持固定。任务完成后,经验证可复用的规则可被纳入未来的 KB 快照。

审计轨迹及其与 SFR_{sem} 的关系。每次调用 V_{sem} 都会记录其判定、适用规则标识符和代码位置。这些自动报告驱动候选过滤,并作为轨迹证据保留,但不会直接用作所报告的 SFR_{sem} 标签。为实现可比评估,三位专家依据相同的规则来源独立审计每个候选,并以多数票决定候选的最终语义标签。只有当一轮中的每个已审计候选都收到无效标签时,该轮才计为语义失败。 N_{sem}^- 统计此类轮次,而 $\text{SFR}_{\text{sem}} = N_{\text{sem}}^- / N_{\text{iter}}$ 。因此,搜索阶段的报告使 NOVA 的自动判定可供审计,而专家标签则为不同方法提供了统一的评估标准。

A.3 可复现性产物

由于生产提示词语料库包含专有操作规则和标识符,我们不会逐字发布它。作为替代,我们通过四类制品支持机制层面的可复现性: (i) 用户任务模板 (§A.3.1), 展示 Solution Design 前解析的输入; (ii) 提示词模板骨架 (§A.3.2), 展示智能体如何使用结构化指导 z_t ; (iii) 语义规则更新和轨迹片段 (§A.3.3), 展示新确认的错误如何影响后续验证; 以及 (iv) 代表性技能摘要 (§A.3.4), 说明特定阶段的过程分解。这些制品共同揭示 NOVA 的核心机制,同时不披露专有内容。

A.3.1 用户任务模板. 每个 NOVA 任务都始于一个结构化用户请求,其中固定了基础模型、带阈值的目标指标、资源预算和数据窗口。借助该模板,主智能体可在方案设计(第 4.6 节)之前将任务路由至正确的能

表 6: V_{sem} 使用的代表性语义规则，取自第 4.3 节所述的三个来源。规则标识符已经匿名化。

来源	作用域	条件（违反时触发 reject）	输出的诊断信息
论文要求	特征-token 映射	分配给同一 token 的特征必须具有相同的语义；添加或移动特征时必须保持论文指定的分组。	规则 ID 及受影响的特征或 token
此前轮次的错误	辅助损失路由	每个辅助损失必须使用其设计所指定目标的预测和标签索引（表 3）。	规则 ID 及受影响的损失、头或索引
KB 规则	序列交互	当需要物品间交互时，Transformer 序列块不得用无法在物品之间交换信息的逐 token MLP 替代自注意力。	规则 ID 及受影响的模块或代码位置

力级别，无需任何后续对话。下面给出一个精简的 L3 示例；其中标识符和路径均已匿名化。这里， τ （提前停止/主要成功阈值）和 R_{max} （每个任务的最大轮数）与正文配置一致， ε 是任务级保护裕量（任何其他指标允许的最大降幅）。

用户任务模板（L3，Literature-to-Production 请求）

```
# 任务
基础模型 : <topo_id>    资源: <group_ids>

# 要求
论文 : <arXiv 链接 >
目标 : 将任务-token 解耦 (TIM) 适配到
      已部署的 RankMixer 主干网络。
窗口: <data_window>

# 指标
主要指标:  $\Delta\text{AUC}_{\text{pp}}^{(0)} \geq \tau$ , 作用于  $\geq 2$  中的
          {og_a, ..., og_d}, 包括 og_a
保护指标 : 其他任何 og 的降幅不得超过  $\varepsilon$ 

# 搜索预算
 $R_{\text{max}}$  (每个任务的最大轮数)

# 约束
实现/资源/工作流约束...
```

A.3.2 提示模板骨架. 下面我们复现 NOVA Solution Design 提示词的一个演示性框架，其中专有标识符和任务特定内容已被隐去。方案生成提示词与 V_{sem} 审查器提示词共用这一模板，仅角色/任务槽不同。它遵循第 4.2 节中的接口：Main Agent 提供所选能力级别和结构化输入，Solution Design 返回 K 个修改方案供 Code Generation 使用。语义验证和候选排序使用不同的 Quality Review 提示词。

Solution Design Agent 提示词

```
# 角色
NOVA 中的 Solution Design 智能体: 每轮接收
Main Agent 选定的能力级别  $\ell \in \{L1, \dots, L4\}$ 
以及先前阶段的输出，调用匹配的技能，
并返回  $K$  个经过排序的修改方案供 Code Generation 使用。

# 输入
 $e_{\text{prev}}$ : 先前的修改方案
 $V = \{\text{sem}, \text{loc}, \text{failure\_loc}\}$ :
      语义/本地/故障定位报告
 $\Delta J$ : 指标增益集合
      { $\Delta\text{AUC}, \Delta\text{DelayAUC}, \Delta\text{Params}, \Delta\text{FLOPs}$ }
 $H$ : 轨迹记忆 (SUMMARY + 先前的 EVALUATION)

# 输出
头部  $z = \{\text{weak\_components}, \text{directions}, \text{forbidden}\}$ 
+ DESIGN/design.md, 其中包含  $K$  个候选方案
(目标模块、预期  $\Delta J$ 、护栏、可运行 diff);
每个方案必须满足  $\Omega$ , 并避开所有禁用方向。

# 迭代
对于  $t > 0$ , 读取先前的 EVALUATION 和累积 SUMMARY,
并改进方向, 而非从头重新生成。
```

A.3.3 一次语义规则更新与一个轨迹片段. 面板 (a) 展示了案例 1 中确认的语义规则，面板 (b) 展示了相应的轨迹证据如何使该规则可供后续轮次使用。该规则属于活跃语义规则集合 $\mathcal{L}_t$ ；它不同于 z_t^{forbid} 中任务范围内的无效模式。

(a) 第 4 轮确认的语义规则

```
{
  rule_id   : "auxiliary-loss-objective-routing",
  source    : " 先前轮次中确认的错误",
  scope     : " 辅助损失路由",
  condition : " 预测索引和标签索引必须
              与设计指定的目标匹配"
}
```

(b) 轨迹摘录 (第 3-5 轮)

```
# t = 3
 $c_{3,j_3^*}$  通过活跃规则检查和本地测试。
→ 离线结果表明,  $o_i$  特定的损失
    可能使用了不匹配的预测/标签索引。

# t = 4
代码检查确认了目标索引错误。
→ 实现得到修正, 并从已确认的错误中
    推导出 (a) 中的语义规则。
→ 候选报告和结果保留在
     $O_5$  中。

# t ≥ 5
 $\mathcal{L}_t$  包含新的辅助损失路由规则。
→  $V_{\text{sem}}(c_{t,j}; \mathcal{L}_{t,j})$  淘汰后续
    重复这一已确认语义错误的候选方案。
```

A.3.4 技能库示例. NOVA 将架构演进分解给特定阶段的智能体, 由它们调用可复用且以文件为依据的技能。下面展示两个代表性技能的精简摘要。每份摘要均保留操作结构—输入、核心 workflows、代表性防护规则和输出—, 同时省略冗长的规则列表与专有字段名。

Material Understanding 技能 (Initialization 阶段)

```
# 输入
任务请求、论文 PDF、生产骨干、目标框架。

# 流程
1. 提取架构、公式、形状和模块依赖关系。
2. 将每一项标记为论文陈述 / 推断 / 工程假设;
    将论文模块与骨干集成点对齐。
3. 输出结构化规范, 并记录未解决的歧义。

# 输出
{material_brief, architecture_map, equation_map,
 shape_constraints, paper_code_alignment,
 ambiguity_log}.md
```

Architecture Planning 技能 (Solution Design)

```
# 输入
 $q, A_t, z_t, KB, \Omega, \ell, K$  (+ material 制品)。

# 流程
1. 将  $q$  置于  $A_t$  的依据之上。
2. 生成  $K$  个不同方案, 并使用  $z_t$  和  $KB$ 。
3. 附上组件、步骤、效果、假设与
    检查项; 排除  $z_t^{\text{forbid}}$  模式。

# 输出
 $(e_{t,1}, \dots, e_{t,K})$ ; 实现与排序仍在
下游进行。
```

A.4 知识库组织与增量构建

NOVA 为特征编辑规范、序列建模、基础特征建模及其他架构修改任务维护任务专用知识库。主智能体检索相关知识库, 以构建搜索期间使用的固定快照 KB 。

内容与构建分离。每个知识库都将可查询的 db_content 与其构建工作流和 db_construction 中的证据分离。对于新代码, 构建技能会追踪配置和执行过程, 对齐特征声明及其用途, 并将实现映射到现有分类体系。只有真正新颖的建模模式或经验证的变体才会提升至 db_content ; 否则复用现有条目, 并将分析保留为构建证据。这样既能保持检索简洁, 又能使未来更新可审计。

序列建模示例。序列建模知识库将特征配置与建模模式、集成范式和经验证的代码案例关联起来。目前其中包含六种序列配置模式和十种建模模式。表 7 和表 8 通过 P02 与 P10 展示了其粒度: P02 描述序列辅助信息不同池化方式, P10 描述序列表示与非序列主干 token 之间的 MixFormer 风格交互。

表 7: 用于序列侧信息池化的代表性 P02 变体。B、L、D、N_f 和 M 分别表示批大小、序列长度、输入宽度、侧信息字段数量和保留的时间分段数量。在 P02d 中，N 个输入分片被归并为 G 个语义组，并投影到宽度 d。

变体	聚合操作	输出形状
P02a：步内归约	在每个时间步独立聚合辅助信息字段。	保留序列长度： $[B, L, N_f \times D] \rightarrow [B, L, D]$ 。
P02b：掩码时间池化	沿时间轴应用带掩码的求和或均值池化。	生成一个序列向量： $[B, L, D] \rightarrow [B, D]$ 。
P02c：分段窗口池化	分别对非均匀时间窗口进行池化。	以 M 个 token 的形式保留粗粒度时间顺序： $[B, L, D] \rightarrow [B, M, D]$ 。
P02d：语义组归约	在语义组内聚合对齐的辅助信息字段，并对分组表示进行投影。	在 G 个组中保留序列长度： $\{[B, L, D]\}_{n=1}^N \rightarrow [B, L, GD] \rightarrow [B, L, d]$ 。

表 8: 用于序列表示与非序列骨干网络 token 之间 MixFormer 风格融合的代表性 P10 变体。

变体	交互设计	关键特性
Heavy MixFormer	使用非序列 token 作为查询，以变长序列表示作为键和值，并采用 FlashAttention 和 grouped-query attention。	具备高容量交互能力，但工程复杂度最高。
Lightweight UniFormer	用在填充序列上执行的手动注意力替换复杂的注意力路径。	更简单的单序列注入。
Dual-sequence gated XDUniFormer	通过并行注意力路径对多种序列类型建模，并利用门控组合其上下文。	支持采用自适应融合的异构序列。
Dual-stage MixFormerV2	在与骨干网络 token 进行交叉注意力之前细化序列表示。	在注入骨干网络之前增加序列侧交互。
Dense MixFormer	将手动注意力与轻量归一化、稠密前馈块和输出融合相结合。	具有中等工程成本的紧凑静态序列实现。